

CI/CD Integrated Pipeline Documentation

A guide to the automated build and testing workflow for the OpenCart project.

1. Overview: The Continuous Quality Loop

The goal of this Continuous Integration and Continuous Deployment (CI/CD) pipeline is to **automate the build, testing, and reporting process** for every code change. This ensures immediate verification, minimizes integration conflicts, and maintains a high standard of code quality across the project.

2. Pipeline Architecture & Triggering

Our pipeline is built using **GitHub Actions**, providing native integration with the repository.

Pipeline Triggers

The workflow is automatically executed under the following conditions:

- **Code Push:** On every push event to the main branch.
- **Pull Request:** On every pull_request targeting the main branch.

Stages Breakdown

The workflow is broken down into five sequential, verifiable steps running on the ubuntu-latest runner:

#	Stage Name	Description	Key Command / Action
1	Checkout	Retrieves the latest source code from the repository.	actions/checkout@v3
2	Setup Java	Installs the necessary Java Development Kit (JDK 17).	actions/setup-java@v3
3	Build	Compiles the project using Maven, skipping	mvn -B package -DskipTests

		tests initially.	
4	Test	Executes the full TestNG test suite against the compiled code.	mvn test
5	Report	Generates and uploads the test results as accessible artifacts.	actions/upload-artifact@v3

3. Pipeline Configuration (.github/workflows/maven.yml)

To activate the automated workflow, create a file named `.github/workflows/maven.yml` with the following YAML content:

```
name: Java CI with Maven
```

```
on:
```

```
  push:
```

```
    branches: [ "main" ]
```

```
  pull_request:
```

```
    branches: [ "main" ]
```

```
jobs:
```

```
  build:
```

```
    runs-on: ubuntu-latest
```

```
    steps:
```

```
      - uses: actions/checkout@v3
```

```
      - name: Set up JDK 17
```

```
        uses: actions/setup-java@v3
```

```
        with:
```

```
          java-version: '17'
```

```
          distribution: 'temurin'
```

```
          cache: maven
```

- name: Build with Maven
run: mvn -B package --file pom.xml -DskipTests
- name: Run Tests
run: mvn test
- name: Upload Test Report
if: always()
uses: actions/upload-artifact@v3
with:
 name: surefire-reports
 path: target/surefire-reports/

4. Quick Start Guide

1. **Project Setup:** Ensure your project contains a standard pom.xml file in the root directory.
2. **Commit Workflow:** Commit the .github/workflows/maven.yml file to your repository's main branch.
3. **Monitor:** Navigate to the **Actions** tab in your GitHub repository to observe the pipeline execution status in real-time.

5. Key Benefits

- **Early Bug Detection:** Automated testing on every commit catches regressions and bugs before they merge.
- **Environment Consistency:** The build process runs in a clean, standardized ubuntu-latest environment every time, eliminating "it works on my machine" issues.
- **Instant Visibility:** The team receives immediate feedback on the build status (Pass/Fail), promoting rapid response to failures.